

Архитектура персональных контекстных движков: Комплексное руководство по реализации RAG для истории чатов Telegram (Редакция 2026 года)

1. Введение: Эволюция персональной информатики и императивы 2026 года

К 2026 году парадигма взаимодействия с персональными данными претерпела фундаментальные изменения. Если в начале десятилетия основные усилия были сосредоточены на создании общих больших языковых моделей (LLM), то сейчас фокус сместился на создание персонализированных систем, способных работать с уникальным контекстом пользователя. История переписки в мессенджерах, таких как Telegram, представляет собой один из самых богатых, но в то же время самых сложных источников данных о жизни человека. Это цифровой слепок, содержащий не только фактологическую информацию, но и эмоциональный контекст, социальные графы, финансовые обязательства и темпоральные планы.

Реализация системы Retrieval-Augmented Generation (RAG) для такого типа данных требует выхода за рамки стандартных архитектурных паттернов "векторизация — поиск — генерация". Чаты — это не статические документы; это потоки сознания, характеризующиеся высокой степенью зашумленности, нелинейностью повествования и имплицитным контекстом. Пользовательский запрос "Сколько я должен Саше?" требует не семантического поиска похожих фраз, а детерминированного извлечения транзакций и их агрегации. Запрос "Когда у нас встреча?" требует понимания относительного времени ("завтра", "на следующей неделе") и привязки к метаданным сообщений.

Данный отчет представляет собой исчерпывающее техническое руководство по проектированию и внедрению системы RAG для анализа истории чатов Telegram, экспортированной в JSON. Мы рассмотрим выбор стека технологий, архитектуру баз данных (SQLite против PostgreSQL), стратегии препроцессинга данных и методы гибридного поиска, объединяющие векторную семантику со строгим SQL-запросом.

1.1 Специфика Telegram-данных как источника знаний

В отличие от корпоративных баз знаний или Википедии, данные Telegram обладают рядом специфических характеристик, которые делают стандартные RAG-подходы

неэффективными без глубокой адаптации:

1. **Высокая фрагментарность:** Средняя длина сообщения в мессенджере составляет 5-15 слов. Векторизация таких коротких фрагментов часто приводит к потере семантического смысла. Вектор фразы "Ок, договорились" практически бесполезен без контекста предыдущего сообщения.
2. **Нелинейная структура:** Механизм `reply_to_message_id` создает древовидную структуру внутри линейной хронологии. Смысловая связь может быть растянута на сотни сообщений, если пользователи активно общаются в групповом чате.
3. **Мультимодальность:** Современное общение — это смесь текста, голосовых сообщений, фотографий чеков, геолокаций и опросов. Игнорирование нетекстовых модальностей приводит к потере критически важной информации (например, фото расписания или скриншот перевода долга).
4. **Смещение интенгов:** В одном диалоге могут переплетаться обсуждение планов на выходные, рабочие вопросы и обмен мемами. Система должна уметь фильтровать "шум" и выделять "сигналы".¹

2. Стратегия приема и предобработки данных (Data Ingestion & ETL)

Фундаментом любой высокопроизводительной RAG-системы является качество конвейера обработки данных (ETL). Работа с `result.json` из Telegram требует многоступенчатого процесса очистки, обогащения и структурирования.

2.1 Анатомия Telegram JSON экспорта

Стандартный экспорт Telegram представляет собой вложенную структуру. Корневой объект содержит метаданные чата (ID, тип, название), а массив `messages` хранит последовательность сообщений.

Поле JSON	Описание и Значение для RAG
<code>id</code>	Уникальный идентификатор сообщения. Критичен для ссылочной целостности графа ответов.
<code>date_unixtime</code>	Временная метка. Основа для темпоральной фильтрации и разрешения относительных дат.
<code>from / from_id</code>	Отправитель. Необходим для атрибуции

	высказываний и разделения "моих" долгов от "чужих".
reply_to_message_id	ID родительского сообщения. Ключ к восстановлению контекста.
text	Содержание. Может быть строкой или массивом объектов (включая форматирование, ссылки, хэштеги).
file / photo	Ссылки на медиа. Требуют OCR или транскрипции.

2.2 Проблема контекстного окна и линеаризация графа

Одной из главных проблем при индексации чатов является потеря контекста. Если пользователь В отвечает пользователю А через 3 часа сообщением "Я согласен", RAG-система, индексирующая сообщения по отдельности, никогда не поймет, с чем именно согласился пользователь.

В 2026 году стандартом является техника **Контекстной Линеаризации (Contextual Linearization)**.³

Алгоритм обогащения контекста:

1. **Построение локального индекса:** Перед обработкой скрипт создает хэш-карту всех сообщений в чате по их id.
2. **Рекурсивное разрешение:** Для каждого сообщения проверяется наличие поля reply_to_message_id. Если оно присутствует, извлекается текст родительского сообщения.
3. **Синтез чанка:** Текст родителя препендингуется (добавляется в начало) к тексту ответа с явным указанием ролей.

Пример трансформации:

Исходный JSON:

JSON

```
{  
  "id": 105,  
  "reply_to_message_id": 42,  
  "text": "Перевел 5000",  
  "from": "Alex"  
}
```

Обогащенный текст для эмбединга:

[Контекст: User A (сообщение 42): Скинь мне долг за ужин.] -> [Ответ: Alex (сообщение 105): Перевел 5000]

Такой подход гарантирует, что векторное представление сообщения будет находиться в семантической близости к понятиям "долг", "ужин" и "возврат средств", а не просто "перевод".⁵

2.3 Сегментация на сессии (Session-Based Chunking)

Простое разбиение на чанки по количеству токенов (например, каждые 500 токенов) разрушает логику диалога. Более эффективным методом является **семантическая сегментация по времени**.

Эвристика "Time-Gap", ставшая стандартом де-факто:

Если разница во времени между сообщением M_i и M_{i+1} превышает пороговое значение ΔT (обычно 15–30 минут), текущий чанк закрывается, и начинается новый. Это позволяет группировать сообщения в логические "сессии" или "разговоры", которые, как правило, посвящены одной теме.

Каждый такой сессионный чанк снабжается метаданными:

- start_time: Время начала сессии.
- end_time: Время окончания.
- participants: Список участников сессии.
- topic_summary: (Опционально) Сгенерированное малой LLM краткое резюме темы сессии.

2.4 Обработка мультимедийных данных

Для полноценного поиска информации о расписаниях и долгах нельзя игнорировать изображения. Часто расписание занятий присылается в виде фото, а подтверждение перевода — скриншотом.

В пайплайне 2026 года обязательным этапом является использование локальных

моделей Vision-Language (например, миниатюрных версий LLaVA или Qwen-VL) или классического OCR (Tesseract 6.x) для извлечения текста из изображений, ссылки на которые есть в JSON. Извлеченный текст добавляется в поле content индексируемого документа с пометкой ``.⁶

3. Архитектура хранения данных: SQLite vs. PostgreSQL

Выбор базы данных — это стратегическое решение, определяющее производительность, сложность поддержки и портативность системы. В контексте анализа *личных* архивов (обычно от 100 тыс. до 5 млн сообщений) выбор стоит между легковесностью SQLite и мощностью PostgreSQL.

3.1 PostgreSQL + pgvector: Корпоративный стандарт

PostgreSQL с расширением pgvector является доминирующим решением для энтерпрайз-систем RAG.

Преимущества:

- **Масштабируемость:** Поддержка индексов HNSW (Hierarchical Navigable Small Worlds) позволяет выполнять поиск по миллионам векторов за миллисекунды.
- **Параллелизм:** Архитектура MVCC (Multi-Version Concurrency Control) идеально подходит для ситуаций, когда идет постоянная запись новых сообщений и одновременное чтение аналитикой.
- **Гибридный поиск:** Мощнейшие возможности полнотекстового поиска (FTS) через tsvector, которые можно комбинировать с векторным поиском в одном SQL-запросе.⁷
- **Экосистема:** Готовые интеграции с облачными провайдерами (Supabase, AWS RDS).

Недостатки для персонального использования:

- **Операционная сложность:** Требуется запуск отдельного сервера (обычно через Docker), настройки портов, управления пользователями и паролями. Это создает "трение" для локальных инструментов.
- **Ресурсоемкость:** Потребляет значительно больше оперативной памяти в простое по сравнению с встраиваемыми решениями.⁹

3.2 SQLite + sqlite-vec: Выбор 2026 года для персональных задач

К 2026 году экосистема SQLite совершила качественный скачок с появлением и стабилизацией расширения sqlite-vec (наследника sqlite-vss). Это расширение, написанное на чистом C, позволяет хранить и искать векторы непосредственно внутри

файла базы данных.¹⁰

Почему SQLite выигрывает в контексте Telegram-истории:

1. **Архитектура "Один файл":** Вся база знаний — чаты, векторы, метаданные, индексы — хранится в одном файле .db. Это обеспечивает идеальную портативность. Вы можете скопировать этот файл на флешку, зашифровать его, перенести на другой ноутбук без необходимости дампа/восстановления сервера.¹²
2. **Производительность "Brute-Force":** Для объемов данных, типичных для личной переписки (даже за 10 лет это редко превышает 1–2 миллиона векторов), полный перебор (brute-force scan) в sqlite-vec работает на удивление быстро благодаря оптимизациям SIMD. Время отклика остается в пределах 50–100 мс, что абсолютно приемлемо для чат-бота.¹³
3. **Local-First философия:** SQLite идеологически соответствует тренду на приватность и локальное хранение данных, исключая передачу информации в облачные управляемые базы данных.

Рекомендуемая схема данных (SQLite):

Для реализации требований по поиску долгов и расписания предлагается гибридная схема:

Таблица	Назначение	Ключевые поля
raw_messages	Хранение полного JSON и метаданных.	id (PK), chat_id, json_blob, timestamp, sender_id
vec_chunks	Виртуальная таблица для векторного поиска (sqlite-vec).	rowid, embedding (BLOB), chunk_text, source_msg_id
structured_debts	Извлеченные факты о долгах.	id, debtor, creditor, amount, currency, due_date, status, evidence_msg_id
structured_events	Извлеченные календарные события.	id, event_title, start_time, end_time, location, evidence_msg_id
fts_idx	Индекс полнотекстового поиска (FTS5).	text_content, msg_id

Эта схема реализует паттерн **Polyglot Persistence** в рамках одной СУБД: векторы для семантики, реляционные таблицы для строгих фактов и FTS для поиска по ключевым словам.¹⁴

3.3 Вердикт по выбору БД

Для задачи "RAG для истории чатов Telegram" в 2026 году **SQLite с расширением sqlite-vec** является оптимальным выбором. Она обеспечивает достаточную производительность, максимальную простоту развертывания и соответствует требованиям приватности. Миграция на PostgreSQL имеет смысл только в том случае, если вы планируете предоставлять этот сервис как SaaS для тысяч пользователей одновременно.¹⁰

4. Инструментарий оркестрации: LlamaIndex против LangChain

Выбор фреймворка для связывания LLM, базы данных и логики приложения является критическим. В 2026 году "война фреймворков" привела к четкой специализации инструментов.

4.1 LangChain: Универсальный агент

LangChain остается мощнейшим инструментом для создания сложных агентов, способных выполнять действия во внешнем мире (поиск в интернете, вызов API). Его архитектура построена вокруг "цепочек" (Chains) и графов выполнения (LangGraph).

Недостатки для данной задачи: LangChain часто критикуют за излишнюю абстракцию и сложность при работе с глубоко структурированными данными. Для задачи, которая на 90% состоит из "Retrieval" (поиска) и анализа данных, LangChain может оказаться избыточным и сложным в отладке.¹⁶

4.2 LlamaIndex: Data-Centric фреймворк

LlamaIndex (ранее GPT Index) изначально создавался с фокусом на "умное хранение и извлечение данных". К 2026 году он стал стандартом де-факто для сложных RAG-систем.¹⁸

Ключевые преимущества LlamaIndex для Telegram RAG:

1. **Продвинутые структуры индексов:** Поддержка иерархических индексов, где сводка чата ссылается на конкретные чанки. Это позволяет LLM сначала найти нужный месяц или тему, а затем углубиться в детали.¹⁹

2. **Нативная поддержка SQL:** LlamaIndex обладает лучшими в классе инструментами для Text-to-SQL (NLSQLTableQueryEngine). Это критически важно для реализации поиска по долгам и расписанию.²⁰
3. **Router Query Engine:** Встроенный механизм маршрутизации запросов, который может автоматически решать, какой инструмент использовать — векторный поиск или SQL-запрос, основываясь на вопросе пользователя.²²
4. **Property Graphs:** Поддержка графовых индексов, позволяющих строить связи между сущностями (Люди, События) поверх текста.²³

Рекомендация: Использовать **LlamaIndex** в качестве основного оркестратора. Его архитектура "Data Agents" идеально ложится на задачу гибридного поиска по структурированным (долги) и неструктурированным (диалоги) данным.

5. Реализация поиска: Гибридная маршрутизация (Router Architecture)

Простой RAG (векторный поиск) не способен надежно ответить на вопрос "Сколько всего денег я одолжил в марте?". LLM плохо считает, а векторный поиск может вернуть не все транзакции. Решение — **гибридная архитектура с маршрутизатором**.

5.1 Концепция Router Query Engine

Система должна состоять из нескольких специализированных "движков" (Query Engines), объединенных под управлением Маршрутизатора (Router).

1. **Vector Query Engine:** Отвечает за семантические, нечеткие вопросы ("О чем мы говорили прошлым летом?", "Какой рецепт пирога скидывала мама?"). Использует sqlite-vec.
2. **SQL Query Engine:** Отвечает за аналитические вопросы и агрегацию ("Сколько я должен?", "Какие встречи на следующей неделе?"). Использует доступ к таблицам structured_debts и structured_events.²⁴

При поступлении запроса, Router (используя легкую LLM) классифицирует интент пользователя и направляет запрос в нужный движок.

5.2 Специфика поиска долгов (Financial Extraction)

Для наполнения таблицы structured_debts используется подход "Extract-Load". В момент приема сообщений (Ingestion) запускается классификатор.

Техническая реализация:

Используется библиотека Pydantic (или Instructor поверх LlamaIndex) для определения

схемы данных:

Python

```
class FinancialTransaction(BaseModel):
    sender: str
    receiver: str
    amount: float
    currency: str = Field(description="ISO code, e.g. RUB, USD")
    transaction_type: str = Field(description="borrow, lend, repay")
    date: str
```

LLM анализирует каждое сообщение на наличие финансовых интенгов. Фразы "скинь мне 500р", "вернул долг", "заплатил за такси" триггерят извлечение данных. Извлеченные объекты валидируются и записываются в SQL-таблицу.

При запросе пользователя "Мои долги", SQL-движок генерирует запрос: `SELECT sum(amount) FROM structured_debts WHERE receiver='Me' AND transaction_type='borrow'` Это дает точный, математически верный ответ, невозможный при чистом векторном поиске.²⁵

5.3 Специфика поиска расписания (Temporal Extraction)

Работа с расписанием осложняется относительностью времени в чатах. Сообщение "Давай встретимся в следующий вторник" бессмысленно без даты отправки сообщения.

Алгоритм темпорального разрешения:

1. **Извлечение:** LLM получает текст сообщения и *референсную дату* (timestamp сообщения).
2. **Нормализация:** Промпт требует преобразовать все относительные даты ("завтра", "через неделю") в абсолютные форматы ISO-8601 (YYYY-MM-DD).
3. **Хранение:** События сохраняются в таблицу `structured_events` с полями `start_time` и `end_time`.

При запросе "Что у меня на следующей неделе?", система выполняет SQL-запрос с фильтром по диапазону дат: `WHERE start_time BETWEEN date('now') AND date('now', '+7 days')`.²⁷

6. Advanced Retrieval: GraphRAG и Графы знаний

Для запросов высокого уровня, таких как "Как менялись наши отношения с 2024 года?" или "Какие основные темы мы обсуждаем в рабочем чате?", векторный поиск дает слишком фрагментарную картину. В 2026 году лучшей практикой является использование GraphRAG.

6.1 Принципы GraphRAG

GraphRAG строит граф знаний поверх текстовых данных.

- **Узлы (Nodes):** Люди, Места, Проекты, События.
- **Ребра (Edges):** Отношения ("Участвовал в", "Организовал", "Ненавидит").

В отличие от простого графа, GraphRAG использует алгоритмы кластеризации (например, Leiden algorithm) для выделения сообществ узлов и генерации текстовых резюме для каждого сообщества.

6.2 Применение к чатам

Когда пользователь задает обобщающий вопрос, система обращается не к сырым сообщениям, а к иерархическим резюме кластеров. Это позволяет синтезировать ответ, описывающий тренды и паттерны, которые невозможно увидеть при чтении отдельных сообщений. Для реализации можно использовать встроенный PropertyGraphIndex в LlamaIndex, который позволяет комбинировать векторный поиск по узлам с топологическим обходом графа.²³

7. Интерфейс и UX: Выбор Frontend-решения

Сложность бэкенда должна быть скрыта от пользователя. Выбор стоит между готовыми WebUI и кастомными дашбордами.

7.1 Open WebUI

Это мощный, готовый к использованию интерфейс, копирующий UX ChatGPT.

- **Плюсы:** Поддержка голосового ввода, готовая интеграция с Ollama, история чатов, управление пользователями.
- **Минусы:** Сложно кастомизировать отображение специфических данных (таблиц долгов, календарей). Ориентирован на текстовый диалог.

7.2 Streamlit

Python-фреймворк для быстрого создания data apps.

- **Плюсы:** Полный контроль над отображением. Возможность нарисовать

интерактивную таблицу долгов (Pandas DataFrame), график активности в чате или виджет календаря.

- **Вердикт:** Для задачи, включающей визуализацию *структурированных* данных (расписание, финансы), **Streamlit** является предпочтительным выбором. Он позволяет создать не просто чат-бота, а "Аналитическую панель управления личной жизнью".³⁰

8. Приватность и Локальный Инференс (Local Inference)

В 2026 году передача личной переписки в облачные API (OpenAI, Anthropic) считается дурным тоном в кругах энтузиастов приватности. Лучшей практикой является полный **Local Air-Gapped RAG**.

- **Модели:** Использование квантованных моделей (4-bit / 6-bit GGUF) семейства Llama 3 (8B) или Mistral Nemo (12B). Они достаточно умны для RAG и работают на потребительских GPU или даже CPU (через llama.cpp).
- **Сервер инференса:** **Ollama** остается стандартом для локального запуска благодаря простоте API и интеграции с LlamaIndex.
- **Эмбединги:** Локальные модели, такие как nomic-embed-text или bge-m3, обеспечивают качество, сравнимое с облачными аналогами, при нулевой стоимости и полной приватности.³²

9. Дорожная карта реализации

Для построения системы следуйте следующему поэтапному плану:

1. **Phase 1: ETL & Parsing.** Разработка скрипта на Python для парсинга result.json. Реализация логики линейаризации ответов и сегментации сессий. Сохранение очищенных данных в raw_messages (SQLite).
2. **Phase 2: Structured Extraction.** Подключение локальной LLM (Ollama) для прохода по истории и извлечения сущностей Debt и Event. Заполнение таблиц structured_debts и structured_events.
3. **Phase 3: Vector Indexing.** Генерация эмбедингов для текстовых чанков с помощью sqlite-vec. Создание индексов.
4. **Phase 4: Orchestration.** Настройка LlamaIndex RouterQueryEngine. Определение инструментов (SQLTool, VectorTool).
5. **Phase 5: Frontend.** Разработка интерфейса на Streamlit. Реализация вкладок "Чат", "Финансы", "Календарь".

Заключение

Создание RAG-системы для Telegram в 2026 году — это упражнение в гибридной архитектуре. Отказ от наивного векторного поиска в пользу комбинации SQL-аналитики и семантического поиска, использование графовых структур для понимания контекста и строгая ориентация на локальное исполнение (SQLite + Ollama) позволяют создать инструмент, который превращает хаос чат-логов в упорядоченную базу знаний о собственной жизни.

Таблица: Сравнительный анализ БД для персонального RAG

Характеристика	SQLite (sqlite-vec)	PostgreSQL (pgvector)
Сложность настройки	Минимальная (библиотека)	Высокая (сервер, Docker)
Векторная производительность	Достаточная для <1M векторов	Экстремальная (100M+)
Типы индексов	Brute-force, Binary Quantization	HNSW, IVFFlat
Память (RAM)	Низкое потребление	Высокое (кэширование)
Полнотекстовый поиск	FTS5 (хорошо)	tsvector (отлично)
Сценарий использования	Персональный ассистент, локальный анализ	SaaS, многопользовательский доступ

7

Джерела

1. Effective Strategies for Handling Long Chat Histories in RAG-Based Chatbots - Chitika, доступ отримано січня 29, 2026, <https://www.chitika.com/strategies-handling-long-chat-rag/>
2. RAG conversational AI: the complete guide to building advanced AI chatbots - Kairntech, доступ отримано січня 29, 2026, <https://kairntech.com/blog/articles/rag-conversational-ai-the-complete-guide-to-building-advanced-ai-chatbots/>
3. How to Handle Follow-up Questions in RAG-based Chats | by Gregory Zem |

Medium, доступ отримано січня 29, 2026,

<https://medium.com/@mne/how-to-handle-follow-up-questions-in-rag-based-chats-2d8032da207b>

4. Send a Text Message or Reply with Telegram Bot API on New Message (Instant) from Discord API - Pipedream, доступ отримано січня 29, 2026,
https://pipedream.com/apps/discord/integrations/telegram-bot-api/send-a-text-message-or-reply-with-telegram-bot-api-on-new-message-instant-from-discord-api-int_BZsqBPY
5. Understanding message replies in Telegram bot API - Latenode Official Community, доступ отримано січня 29, 2026,
<https://community.latenode.com/t/understanding-message-replies-in-telegram-bot-api/10463>
6. Transform Telegram chat export in JSON to a easy-to-read line-by-line text file. - GitHub Gist, доступ отримано січня 29, 2026,
<https://gist.github.com/piggynl/c146c7d232a36c6897c25246cb35a020>
7. Advice needed: Is PostgreSQL + pgvector good enough for a business directory chat app? : r/vectordatabase - Reddit, доступ отримано січня 29, 2026,
https://www.reddit.com/r/vectordatabase/comments/1p4u8v9/advice_needed_is_postgresql_pgvector_good_enough/
8. Choosing vector database: a side-by-side comparison - Hacker News, доступ отримано січня 29, 2026, <https://news.ycombinator.com/item?id=37764489>
9. SQLite vs PostgreSQL: A Detailed Comparison - DataCamp, доступ отримано січня 29, 2026,
<https://www.datacamp.com/blog/sqlite-vs-postgresql-detailed-comparison>
10. Retrieval Augmented Generation in SQLite | Towards Data Science, доступ отримано січня 29, 2026,
<https://towardsdatascience.com/retrieval-augmented-generation-in-sqlite/>
11. Introducing sqlite-vec v0.1.0: a vector search SQLite extension that runs everywhere - Reddit, доступ отримано січня 29, 2026,
https://www.reddit.com/r/LocalLLaMA/comments/1ehlazq/introducing_sqlitevec_v010_a_vector_search_sqlite/
12. Building a RAG on SQLite, доступ отримано січня 29, 2026,
<https://blog.sqlite.ai/building-a-rag-on-sqlite>
13. You may not need pg_vector, sqlite-vss, etc. - DEV Community, доступ отримано січня 29, 2026,
<https://dev.to/nvahalik/you-may-not-need-pgvector-sqlite-vss-etc-e6j>
14. Is SQLite a sensible option for data logging? - Software Engineering Stack Exchange, доступ отримано січня 29, 2026,
<https://softwareengineering.stackexchange.com/questions/208471/is-sqlite-a-sensible-option-for-data-logging>
15. sqlite-history: tracking changes to SQLite tables using triggers (also weeknotes), доступ отримано січня 29, 2026,
<https://simonwillison.net/2023/Apr/15/sqlite-history/>
16. LlamaIndex vs. LangChain: Which RAG Tool is Right for You? - n8n Blog, доступ отримано січня 29, 2026, <https://blog.n8n.io/llamaindex-vs-langchain/>

17. LlamaIndex vs LangChain: RAG framework differences - Statsig, доступ отримано січня 29, 2026, <https://www.statsig.com/perspectives/llamaindex-vs-langchain-rag>
18. Llamaindex vs Langchain: What's the difference? - IBM, доступ отримано січня 29, 2026, <https://www.ibm.com/think/topics/llamaindex-vs-langchain>
19. Structured Hierarchical Retrieval | LlamaIndex Python Documentation, доступ отримано січня 29, 2026, https://developers.llamaindex.ai/python/examples/query_engine/multi_doc_auto_retrieval/multi_doc_auto_retrieval/
20. How to store chat history with query engine? · run-llama llama_index · Discussion #15146 - GitHub, доступ отримано січня 29, 2026, https://github.com/run-llama/llama_index/discussions/15146
21. Combining Text-to-SQL with Semantic Search for Retrieval Augmented Generation, доступ отримано січня 29, 2026, <https://www.llamaindex.ai/blog/combining-text-to-sql-with-semantic-search-for-retrieval-augmented-generation-c60af30ec3b>
22. Routers | LlamaIndex Python Documentation, доступ отримано січня 29, 2026, https://developers.llamaindex.ai/python/framework/module_guides/querying/router/
23. microsoft/graphrag: A modular graph-based Retrieval ... - GitHub, доступ отримано січня 29, 2026, <https://github.com/microsoft/graphrag>
24. Use a `RouterQueryEngine` as a chat · run-llama llama_index · Discussion #11907 - GitHub, доступ отримано січня 29, 2026, https://github.com/run-llama/llama_index/discussions/11907
25. Recursive Question Understanding for Complex Question Answering over Heterogeneous Personal Data - arXiv, доступ отримано січня 29, 2026, <https://arxiv.org/html/2505.11900v1>
26. Bank support - Pydantic AI, доступ отримано січня 29, 2026, <https://ai.pydantic.dev/examples/bank-support/>
27. Beyond Static Graphs: Engineering Evolving Relationships - Zep, доступ отримано січня 29, 2026, <https://blog.getzep.com/beyond-static-knowledge-graphs/>
28. Enhancing RAG with Time Filters Using Instructor, доступ отримано січня 29, 2026, <https://python.useinstructor.com/blog/2024/06/06/enhancing-rag-with-time-filters-using-instructor/>
29. Memory Graph | GraphRAG, доступ отримано січня 29, 2026, <https://graphrag.com/reference/knowledge-graph/memory-graph/>
30. Open Web UI vs Streamlit (2026) Which One Is BETTER? - YouTube, доступ отримано січня 29, 2026, <https://www.youtube.com/watch?v=51R9YRDIZuI>
31. I built an LLM-powered app to turn text into a calendar events (.ics) : r/Streamlit - Reddit, доступ отримано січня 29, 2026, https://www.reddit.com/r/Streamlit/comments/1lzknx0/i_built_an_llmpowered_app_to_turn_text_into_a/
32. Install GraphRAG Locally: vLLM & Ollama Setup Guide - Chitika, доступ

отримано січня 29, 2026,

<https://www.chitika.com/graphrag-local-install-setup-using-vllm-and-ollama/>

33. GraphRAG local setup via vLLM and Ollama : A detailed integration guide. -

Medium, доступ отримано січня 29, 2026,

<https://medium.com/@ysaurabh059/graphrag-local-setup-via-vllm-and-ollama-a-detailed-integration-guide-5d85f18f7fec>